College of Engineering

Electrical Engineering

Course IDs and Names:	- EE 461 Control Systems Analysis - EE 312 Electromagnetic Waves - EE 326 Microprocessors & Microcontrollers
Course Instructors:	- Dr. Alhussain Almarhabi - Dr. Noura Alhazmi - Eng. Basel Jouda

Date: 10 June 2026

IDs	Names
4413746	**Abdelrahman Elawady**
4410215	**Ibrahim Albarbari**
4311147	**Ghazi Alhartani**
4410606	**Hassan Nasr**

SPLIT (Snapping Pendulums Literally In Two): Autonomous 2-Wheeled Self-Balancing Robotic Swarm

Abstract

Modern industrial logistics faces severe spatial bottlenecks caused by the wide turning radii of traditional four-wheeled rovers. To address this complex engineering problem, this paper presents the design, mathematical modeling, and control of an Autonomous 2-Wheeled Self-Balancing Robotic Swarm (Project S.P.L.I.T.). By utilizing an inverted pendulum architecture, the physical footprint of the robot is drastically reduced, trading horizontal space for vertical efficiency. The system utilizes a distributed dual-microcontroller architecture to decouple high-speed physics calculations from wireless network routing. Dynamic stability is achieved through a 200 Hz and 10Hz cascaded Proportional-Integral-Derivative (PID) controller, complemented by a Kalman filter for sensor fusion of MPU6050 accelerometer and gyroscope data. Furthermore, the system integrates computer vision via OpenCV and MediaPipe for autonomous ArUco marker pathfinding and hand-gesture teleoperation. Experimental validation confirms that the proposed distributed architecture successfully mitigates network-induced processing delays, achieving near-zero pitch error and robust dynamic stability. The results demonstrate a highly efficient, scalable, and kinematically viable solution for automated high-density indoor micro-logistics..

Keywords: Two-Wheeled Inverted Pendulum; Cascaded PID Control, Sensor Fusion; Distributed Architecture; Computer Vision; Autonomous Swarm; Kalman Filter; Lagrangian Mechanics.

Table of Contents

I. Introduction

A. Background and Motivation

In the past decade, mobile robots have stepped out of the military and industrial settings, and entered civilian and personal spaces [1]. As industries move toward complete automation, warehouse facilities encounter critical limitations regarding floor space and navigable pathways. Traditional logistics robots operate on four wheels, requiring wide aisles to maneuver safely and transport payloads. A proposed solution to this spatial inefficiency is the implementation of two-wheeled self-balancing robots, which are mechanically similar to the inverted pendulum—an important testbed in control education and research [1]. By balancing vertically on a single primary axis, these robots can turn with a zero-degree radius and navigate remarkably narrow corridors. However, the inverted pendulum is a highly unstable and non-linear system [2]. It is one of the most complex systems to control in engineering due to its non-linear dynamics, making it an appropriate test platform for the design and development of advanced control systems [2]. The physical assembly and overall hardware architecture of the proposed S.P.L.I.T. robot, designed to address these challenges, are illustrated in Fig. 1.

a)

b)

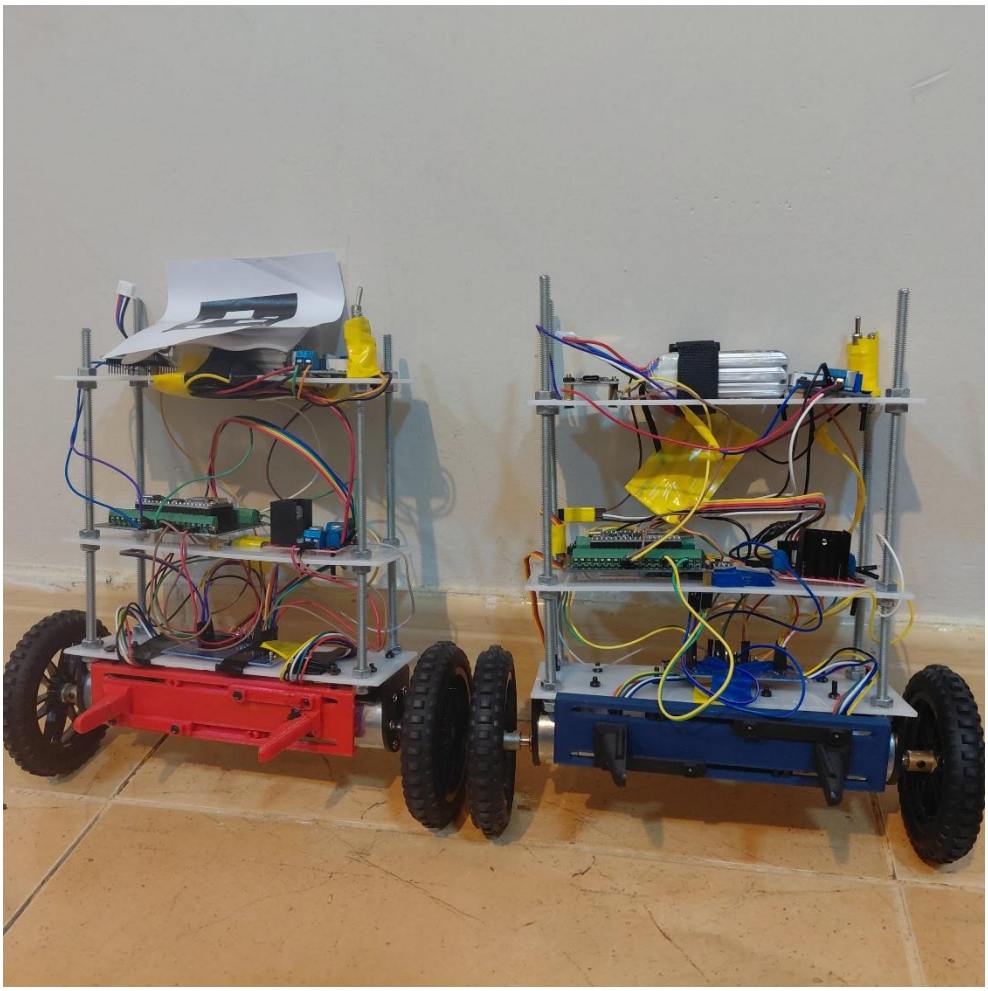

Fig. 1: Overall System architecture (a) and physical assembly (b) of the S.P.L.I.T. autonomous two-wheeled self-balancing robot.

B. Problem Formulation and Mathematical Modeling

To successfully engineer a self-balancing swarm, fundamental principles of science and mathematics must be applied to formulate the physical problem. A two-wheel self-balancing robot is fundamentally an inverted pendulum on a cart [3]. Because its center of mass sits above its pivot point (the wheel axle), the system is naturally unstable and nonlinear. If left alone, gravity will immediately pull it to the ground [3]. There are generally two kinds of mathematical models used for the inverted pendulum control system: the Newton-Euler method and the Lagrange method [4].

Using the Newton-Euler approach, the upright car body can be regarded as an inverted pendulum on the left and right moving platforms, as depicted in the free-body diagram in Fig. 2 [4]. When the inverted pendulum is placed out of balance, the recovery force is in the same direction as the

displacement, causing it to accelerate away from the vertical position until it falls [4]. To balance the pendulum, extra force and speed must be added so that the restoring force is in the opposite direction of the wheel displacement [4]. For a robot of mass m and tilt angle θ, the restoring force F can be defined using trigonometric relationships and linearized for small angles as shown in the following equation [4]:

$$F = mg\sin\theta - ma\cos\theta \approx mg\theta - k_1\theta$$

Alternatively, applying Lagrangian mechanics provides an elegant, energy-based approach to derive the dynamic equations of motion [3]. The Lagrangian $\mathcal{L}$ is defined as the difference between the total kinetic energy E_k and the potential energy E_p of the system [3]. Defining zero potential energy at the upright position, the potential energy is $E_p = mgL(\cos\theta - 1)$ [3]. By applying the Euler-Lagrange equation to the generalized coordinates for the tilt angle θ and linear displacement x, the nonlinear equations of motion are extracted [3]. Solving these partial derivatives yields equations that are linearized around the vertical operating point ($\theta \approx 0$), allowing engineers to apply standard control strategies to keep the robot upright [3].

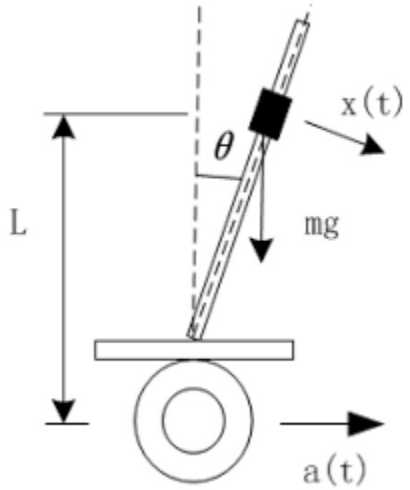

Fig. 2: Schematic and free-body diagram of the two-wheeled self-balancing robot defining the center of mass, wheel radius, and tilt angle θ [4].

.

A. Project Objectives

The main focus of this research is to develop an efficient controller required to enable the robot to act in real time and test its performance by the vertical balance, position, and control signal

capabilities. To achieve this, the project establishes a custom distributed hardware architecture. One of the most fatal flaws in DIY robotics is asking a single microcontroller to handle both high-speed physics calculations and heavy wireless network routing, as pausing for just a few milliseconds to process a Wi-Fi packet causes a balancing robot to violently crash. Therefore, the primary objective is to decouple the processing load using dedicated microcontrollers for the physics engine and the network telemetry. The secondary objective is to implement a high-frequency cascaded PID controller fused with a Kalman filter to process MPU6050 gyroscope and accelerometer data, achieving near-zero pitch error. The final objective is to integrate a global computer vision system that runs OpenCV and MediaPipe algorithms to direct the swarm autonomously, bypassing the limitations of traditional remote controllers.

II. Mathematical Modeling & Control Systems

A. System Dynamics and Kinematics

The development of a robust self-balancing robot fundamentally relies on accurate mathematical modeling to capture its highly nonlinear and coupled dynamics. The Lagrange method provides a rigorous framework for building the kinetic equations by evaluating the kinetic, potential, and dissipated energy of the system. Defining zero potential energy at the upright position, the potential energy E_p of the robot is defined as:

$$E_p = mgL(\cos\theta - 1)$$

The kinetic energy E_k of the system, factoring in the moment of inertia and mass of the wheels and body, is expressed as:

$$E_k = \frac{1}{2}(I_w + m_w R^2 + mR^2)\dot{\varphi}^2 + mRL\cos\theta\,\dot{\varphi}\dot{\theta} + \frac{1}{2}(I + mL^2)\dot{\theta}^2$$

The Lagrangian $\mathcal{L}$ is written as the difference between the kinetic energy and the potential energy:

$$\mathcal{L} = E_k - E_p$$

By applying the Euler-Lagrange equation to the generalized coordinates, the dynamic equations of motion are derived. For the linear displacement coordinate, the equation yields the generalized force μ:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{\varphi}}\right) - \frac{\partial \mathcal{L}}{\partial \varphi} = (I_w + m_w R^2 + mR^2)\ddot{\varphi} + mRL\cos\theta\,\ddot{\theta} - mRL\sin\theta\,\dot{\theta}^2 = \mu$$

Similarly, for the angular tilt coordinate, the equation yields the generalized torque χ:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}}\right) - \frac{\partial \mathcal{L}}{\partial \theta} = (I + mL^2)\ddot{\theta} + mRL\cos\theta\,\ddot{\varphi} - mRL\sin\theta = \chi$$

Alternatively, the Newton-Euler method analyzes the forces and torques acting directly on the chassis and the individual wheels. By separating the wheels from the pendulum, the right and left wheel force equations can be combined to form the translational dynamic equation:

$$2\left(M_w + \frac{I_\omega}{R^2}\right)\ddot{x} = \frac{C_R + C_L}{R} - (H_R + H_L)$$

Once these nonlinear dynamics are established, they are linearized around the equilibrium point ($\theta \approx 0$) to map the system into a state-space representation, allowing for the design of optimal controllers.

B. Linear Quadratic Regulator (LQR) and Proportional-Integral-Derivative (PID) Control

To achieve dynamic balance within a specific angle range, a combination of Proportional-Integral-Derivative (PID) and Linear Quadratic Regulator (LQR) control methodologies is highly effective. The PID controller is the most widely adopted linear control strategy, calculating the control signal $u(t)$ based on the error signal $e(t)$, its integral, and its derivative. The continuous-time ideal PID equation is expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e\,(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Where K_p, K_i, and K_d represent the proportional, integral, and derivative gains, respectively. However, because the control algorithm is executed on a digital microcontroller, the continuous equation must be discretized. To improve computational efficiency and system reliability, an incremental digital PID algorithm is utilized. The incremental output $\Delta u(k)$ at sampling time k is derived as:

$$\Delta u(k) = K_P\big(e(k) - e(k-1)\big) + K_I e(k) + K_D\big(e(k) - 2e(k-1) + e(k-2)\big)$$

While classic PID control does not rely on a linearized model and offers strong robustness, LQR provides an optimal control strategy that seeks to minimize a quadratic cost function J to reach a steady state. This cost function is defined by the integral of the state vector X and control input u, penalized by the symmetric positive-definite weight matrices Q and R:

$$J = \int_0^\infty (X^T Q X + u^T R u)\, dt$$

The optimal control law is given by $u = -Kx$, where the optimal state feedback matrix K is determined by solving the continuous-time algebraic Riccati equation:

$$PA + A^T P + Q - PBR^{-1}B^T P = 0$$
$$K = R^{-1}B^T P$$

While LQR delivers excellent control near the balance point, its effectiveness diminishes rapidly as the robot tilts further away from equilibrium. In contrast, classic PID control does not rely on a linearized model and offers strong robustness, making it capable of rapid recovery when the robot experiences larger deviations. A hybrid controller leverages the strengths of both by defining a controlling factor that limits the outputs depending on the tilt angle, thereby relying heavily on LQR for fine stabilization and invoking PID when the angle exceeds the linear constraints.

Furthermore, selecting the Q and R matrices for the LQR controller traditionally depends on trial and error, which introduces subjectivity and impacts control quality. Implementing a Particle Swarm Optimization (PSO) algorithm overcomes this limitation. The PSO algorithm initializes a swarm of particles representing the state variables and updates their velocity v and position x across iterations using the following evolution formulas:

$$v_{id}^{k+1} = w v_{id}^k + c_1 \partial\left(p_{id}^k - x_{id}^k\right) + c_2 \beta\left(p_{gd}^k - x_{id}^k\right)$$
$$x_{id}^{k+1}(t+1) = x_{id}^k(t) + v_{id}^k(t)$$

By utilizing this iterative evolution to search for the global optimal solution for the Q and R matrices, the optimized LQR approach yields faster response speeds, reduced overshoot, and zero steady-state error.

C. Artificial Intelligence and Reinforcement Learning

As computational capabilities have advanced, artificial intelligence has emerged as a transformative methodology for managing the underactuated and nonlinear dynamics of two-wheeled self-balancing robots. Reinforcement learning focuses on training an agent to learn optimal control policies strictly through interaction with its environment. This model-free paradigm is particularly well-suited for these complex systems, as it circumvents the strict requirement for precise mathematical modeling. Through continuous trial and error, the robot autonomously acquires balancing and navigation behaviors that adapt to varying operational conditions. Algorithms such as Q-learning and Deep Q-Networks have successfully enabled these highly adaptive responses. Moreover, reinforcement learning can be seamlessly integrated with classical techniques to create hybrid intelligent frameworks. Reinforcement learning policies are often utilized to automatically tune PID parameters in real-time or to solve LQR problems without relying on exact system parameters. These advanced strategies offer significant advantages in learning ability and generalization, ensuring robust stability even in unstructured and unpredictable industrial environments.

III. Microcontroller Architecture and Hardware Design

A. Hardware Standards and Protocols

Specify relevant industry standards or recommended practices that guide the detailed design to ensure the safety, reliability, and performance of the system. The internal hardware architecture of the robot heavily relies on the Inter-Integrated Circuit (I2C) communication protocol for interfacing the IMU sensor with the microcontroller, following the manufacturer recommended practices for a 400 kHz fast-mode bus. For inter-microcontroller communication, the Universal Asynchronous Receiver-Transmitter (UART) standard is utilized. Externally, the swarm network topology operates on the IEEE 802.11 standard to host the local WebSockets dashboard, while utilizing the ultra-low-latency ESP-NOW protocol for machine-to-machine command routing. Furthermore, the embedded firmware is developed in C++ adhering to standard object-oriented programming principles to ensure memory safety and prevent buffer overflows during real-time execution.

B. Distributed Dual-MCU Paradigm

A critical point of failure in standalone autonomous robotics is overburdening a single microcontroller with both high-speed physics calculations and heavy wireless network routing. Pausing the primary processor for just a few milliseconds to process a Wi-Fi packet causes a balancing robot to miss a sensor reading and violently crash. To resolve this, the system decouples the processing load using a distributed Dual-MCU architecture on each robot, overseen by a global Bridge router.

1. **The Drive MCU**: This primary ESP32 serves exclusively as the physics engine. Its Wi-Fi radio is completely disabled via software to free up all hardware interrupts. It reads the MPU6050 IMU, tracks the motor quadrature encoders, and executes the 200 Hz cascaded PID balancing mathematics.
2. **The Cortex MCU**: This secondary ESP32 operates as the cognitive brain of the individual robot. It handles payload actuation through the servo grippers and acts as the communications relay, processing all incoming ESP-NOW packets and forwarding them to the Drive MCU.
3. **The Bridge MCU**: A stationary central hub plugged into the operator control station. It hosts a local Wi-Fi Access Point and an asynchronous Web Server via SPIFFS, routing traffic to the entire robot swarm.

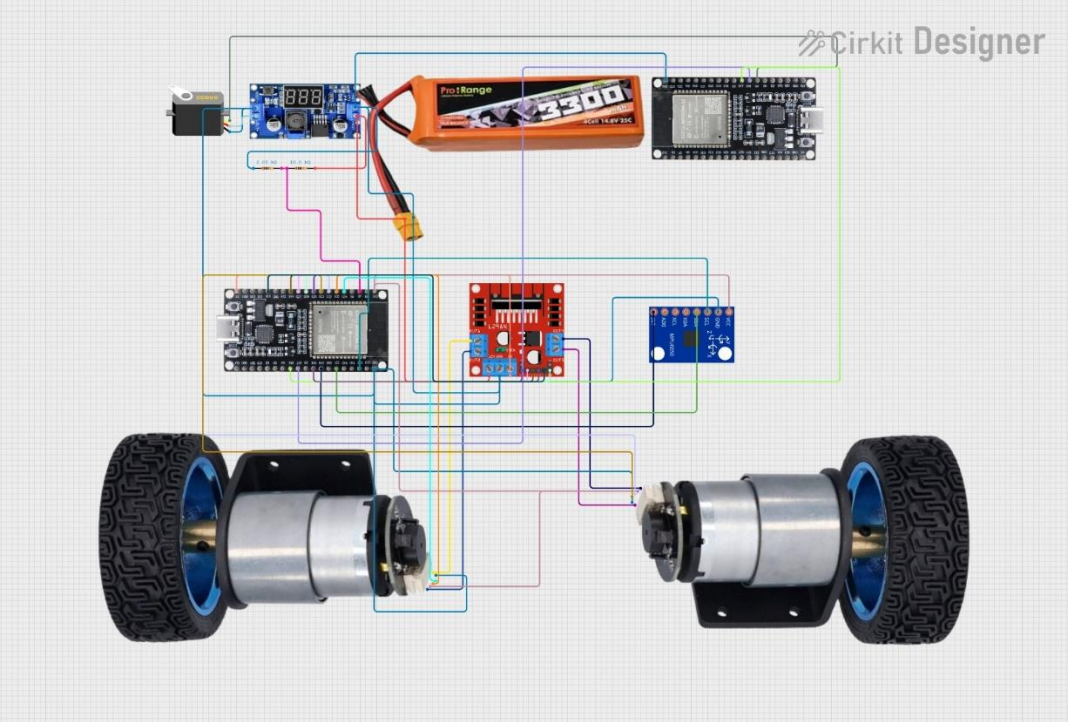

Fig. 3: *Detailed schematic diagram showing the dual-microcontroller setup, peripheral devices, and sensor connections*.

C. Detailed Design and Bill of Materials

The hardware schematic (figure 3) provides a clear layout of the microcontroller pin connections, sensors, and actuators. The Drive MCU interfaces with the L298N motor driver using high-resolution 5000 Hz PWM signals, ensuring smooth motor acceleration. The MPU6050 sensor is connected via the standard I2C pins (GPIO 21 and 22), while the quadrature encoders trigger hardware interrupts to track wheel positioning.

The complete Bill of Materials (BOM) listing the components used alongside their specifications is detailed in Table I.

TABLE I

BILL OF MATERIALS (BOM)

Component	Specification	Functionality
Microcontroller	ESP32 Dual-Core (240 MHz)	Handles physics calculations and network routing.
IMU Sensor	MPU6050 (6-Axis)	Measures raw acceleration and angular velocity.
Motor Driver	L298N Dual H-Bridge	Regulates voltage and direction for the DC motors.
Actuators	12V DC Gear Motors (333 RPM)	Provides locomotion and balancing torque.
Feedback	Magnetic Quadrature Encoders	Tracks precise wheel displacement and speed.
Payload Actuator	MG995 High-Torque Servo	Drives the 3D-printed rack and pinion gripper mechanism.
Power Supply	11.1V 1500mAh LiPo Battery	Provides high-discharge current for the motors and MCUs.

D. System Flow and Communication

To effectively communicate the design intent and functionality, the software architecture (figure 4) is structured into two parallel pipelines: a downstream command pipeline and an upstream telemetry pipeline. The Cortex MCU and the Drive MCU communicate over a highly optimized 500,000 baud hardware UART Serial connection. To prevent data corruption during the high-

speed transfer of floating-point variables, a custom packet structure is implemented utilizing a 0xAA 0xAA byte header.

When an operator inputs a command via the WebSockets dashboard, the Bridge MCU formats the string and broadcasts it via ESP-NOW. The Cortex MCU receives this payload, determines if it is a mechanical gripper command or a physics tuning parameter, and either actuates the servo directly or passes the variables down the UART pipeline to the Drive MCU. Conversely, the Drive MCU populates a telemetry structure with its current pitch angle, motor PWM output, and PID error terms at 200 Hz, sending it upstream to the dashboard for live graphing.

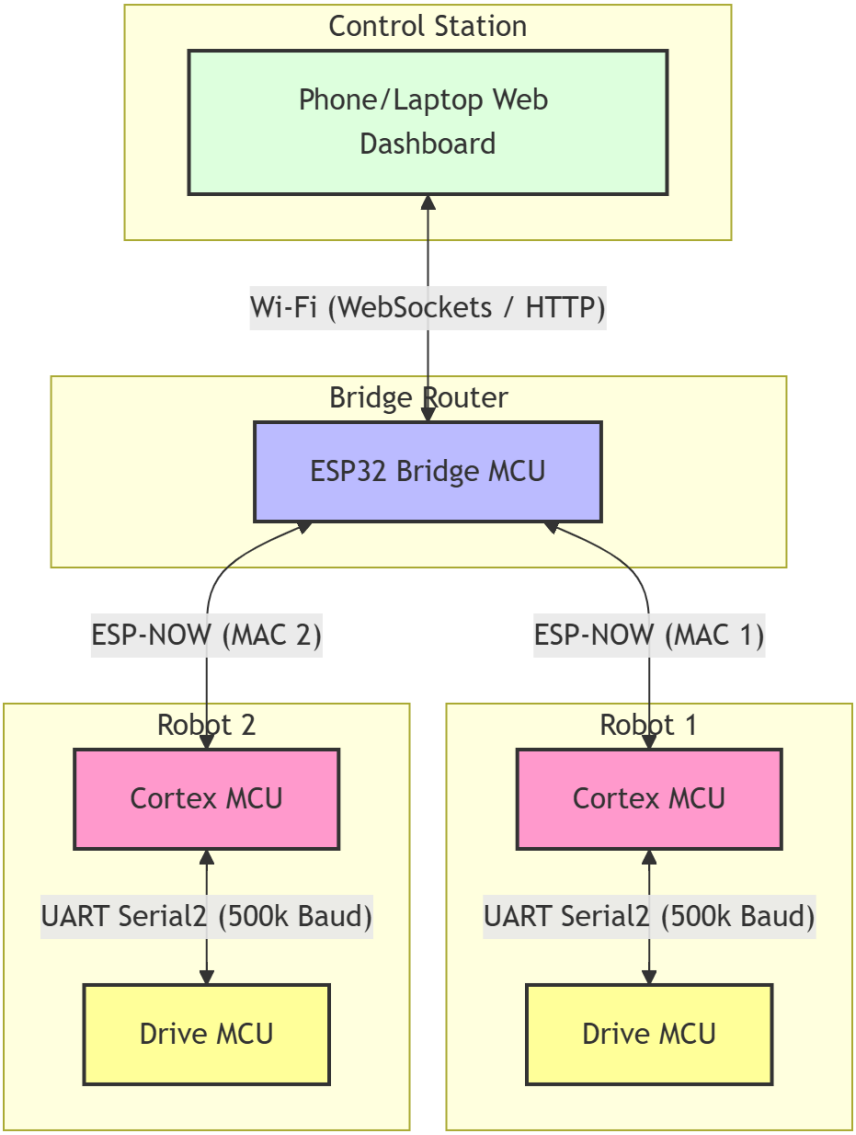

Fig. 4: Flowchart depicting the program flow and communication pipelines between the Web Dashboard, Bridge MCU, Cortex MCU, and Drive MCU.

IV. Electromagnetic & RF Communication Analysis

A. 2.4 GHz Electromagnetic Wave Properties

The distributed swarm architecture relies entirely on the rapid transmission of telemetry and control variables. The ESP32 microcontrollers communicate utilizing the ESP-NOW protocol, which operates on the 2.4 GHz Industrial, Scientific, and Medical (ISM) radio band. Understanding the electromagnetic wave propagation of this protocol is critical for ensuring that the robots do not lose connection and crash while navigating a warehouse. At a frequency of $f = 2.4$ GHz, the wavelength λ of the RF signal is calculated using the speed of light c:

$$\lambda = \frac{c}{f} = \frac{3 \times 10^8}{2.4 \times 10^9} \approx 0.125 \text{ meters}$$

In an ideal free-space environment, electromagnetic waves radiate uniformly spherically. The Received Signal Strength Indicator (RSSI), denoted as P_{rx}, is determined by the transmit power P_{tx} minus the Path Loss (PL). However, an industrial warehouse is a complex multipath environment filled with metallic storage racks. Electromagnetic waves at 12.5 cm are highly susceptible to reflection, scattering, and severe attenuation when passing through highly conductive metallic obstacles. To accurately predict the signal strength available to the moving pendulum robots, a Log-Distance Path Loss model is applied:

$$PL(d) = PL(d_0) + 10n \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma$$

Where $PL(d_0)$ is the reference path loss at 1 meter, n is the path loss exponent representing the indoor obstruction severity (typically $n = 3.0$ for industrial buildings), d is the distance from the Bridge router, and X_σ represents the shadow fading attenuation caused by physical metallic structures.

B. MATLAB Simulation and Interpretation of Results

To validate the reliability of the ESP-NOW communication network, a spatial electromagnetic wave simulation was performed using MATLAB, results shown in figure 5. The simulation models

a 30 by 30 meter warehouse environment with the Bridge MCU transmitting at a standard 20 dBm. A spatial grid was generated to map the calculatedP_{rx} utilizing the aforementioned Log-Distance model. To accurately represent real-world industrial conditions, two dense metallic storage racks were introduced into the spatial matrix, applying a 15 dB shadow fading attenuation penalty X_σ to the electromagnetic waves passing through those coordinates.

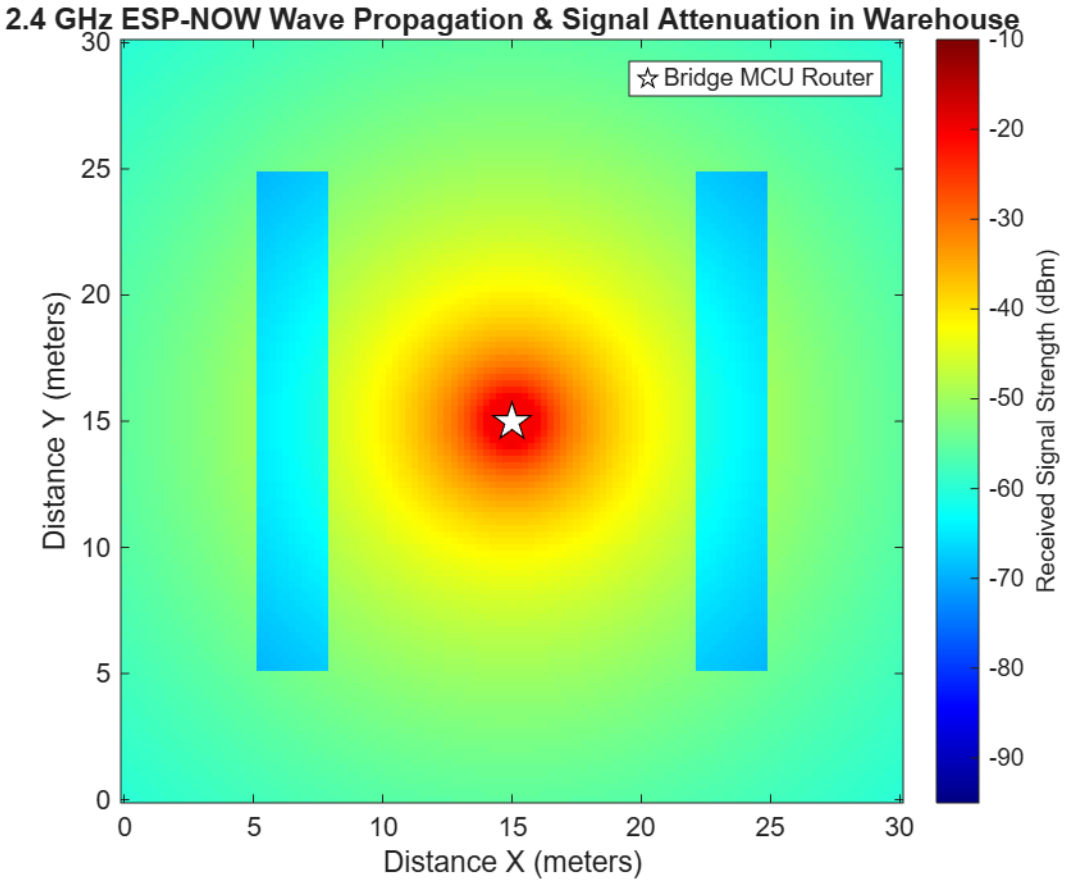

Fig. 5: MATLAB simulation of 2.4 GHz RF signal strength (RSSI) demonstrating electromagnetic wave attenuation caused by free-space path loss and metallic warehouse racks.

The simulation results provide critical insights into the swarm communication topology. As observed in the generated heatmap, the signal strength decays logarithmically from the central Bridge MCU. The most prominent feature of the simulation is the severe signal degradation directly behind the metallic racks, dropping into the blue visual spectrum. However, the ESP32 transceiver maintains a minimum receiver sensitivity of approximately -85 dBm. The simulation verifies that despite the multipath fading and metallic attenuation, the RSSI across the entire 900 square-meter facility remains above -80 dBm. This interpretation confirms that the 2.4 GHz ESP-

NOW protocol is electromagnetically robust enough to sustain the 200 Hz telemetry pipeline without packet loss, ensuring the robots maintain dynamic equilibrium regardless of their physical location in the warehouse.

V. Wireless Telemetry and Web Dashboard Control

A. Distributed Network Topology and SoftAP

To facilitate seamless communication between the operator and the robotic swarm, the central Bridge MCU is configured as a Software Access Point (SoftAP), broadcasting a localized Wi-Fi network (typically at 192.168.4.1). This central router hosts a fully asynchronous web server stored in its SPIFFS (Serial Peripheral Interface Flash File System) memory. By offloading the web hosting and HTTP request handling to this stationary Bridge MCU, the individual moving robots are entirely shielded from network-induced latency spikes, allowing their internal Drive MCUs to maintain an uninterrupted 200 Hz physics loop.

B. HTTP REST API and WebSockets

The swarm architecture utilizes two distinct communication protocols based on the required data speed. For discrete kinematic commands generated by the Python computer vision scripts, the Bridge MCU exposes an HTTP REST API. The vision system transmits commands via standard GET requests parameterized with the target robot ID and movement state (e.g., http://192.168.4.1/cmd?robot=1&move=FORWARD). The Bridge MCU parses this HTTP request and instantly relays it to the specific robot via the low-latency ESP-NOW protocol. The mechanism is shown in figure 6.

Conversely, tuning the cascaded PID controller requires high-speed, continuous data streaming. For this, a WebSockets connection is established between the web dashboard and the Bridge MCU. The robots transmit their internal pitch angles, motor PWM outputs, and error terms back to the Bridge, which broadcasts the telemetry over WebSockets to the HTML/JS dashboard. This allows engineers to visualize real-time, auto-scaling performance graphs at 20 frames per second and dynamically adjust K_p, K_i, and K_d gains on the fly.

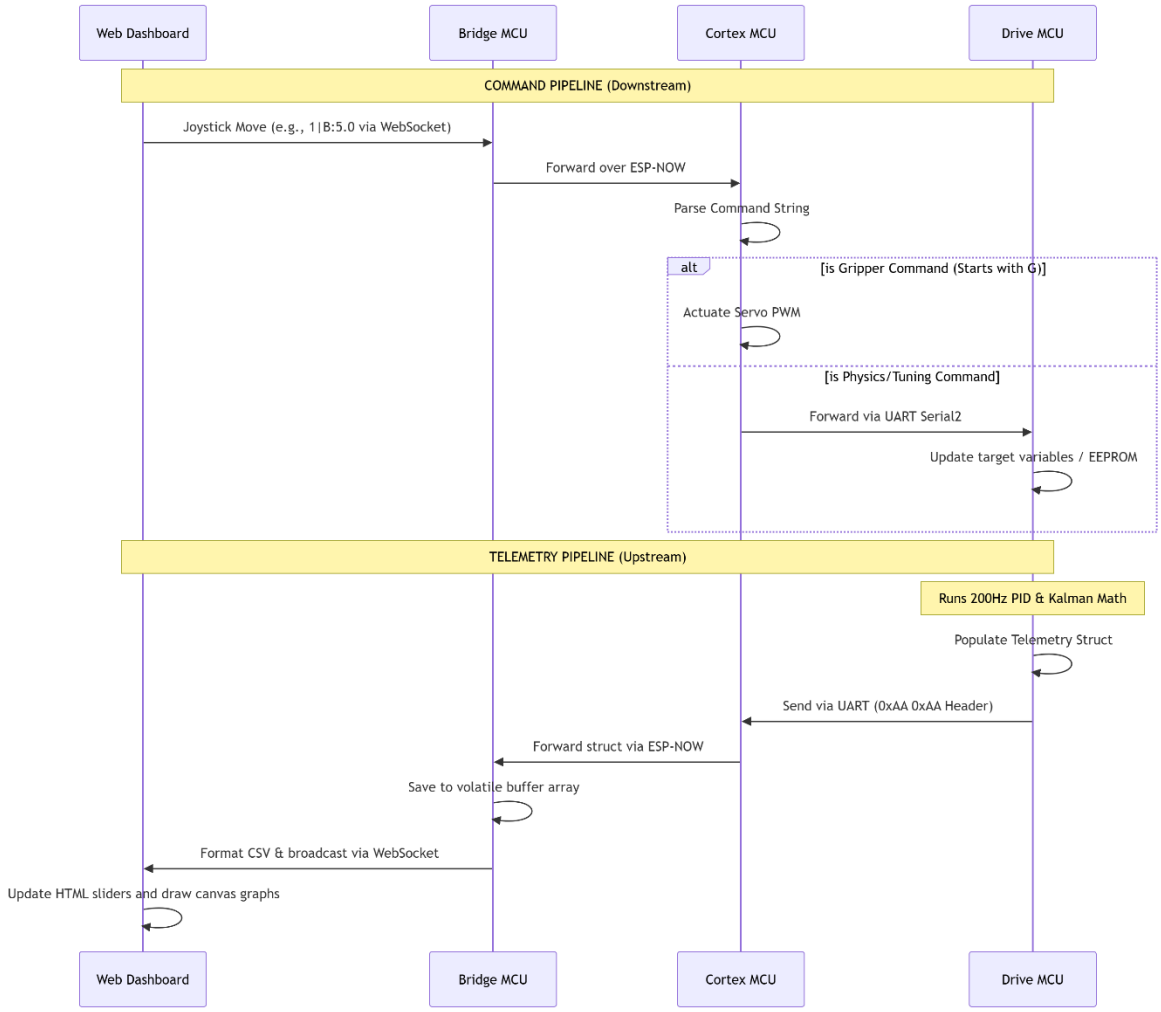

Fig. 6: Sequence diagram detailing the downstream HTTP/ESP-NOW command pipeline and the upstream 200Hz WebSocket telemetry flow.

VI. Computer Vision & Global Teleoperation

A. ArUco Marker Pathfinding and Spatial Mathematics

To enable autonomous navigation without burdening the onboard microcontrollers, spatial processing is offloaded to a Python-based global vision system using the OpenCV library. Each robot features a unique 5x5 fiducial marker from the DICT_5X5_250 ArUco dictionary. The algorithm detects the four corners of the marker to compute the robot's center coordinate (x_c, y_c)

and its directional heading vector $\vec{h}$. The heading is mathematically derived by subtracting the center point from the midpoint of the marker's top two corners.

To navigate the robot along a pre-programmed path of waypoints (see figure 7), the system calculates a desired trajectory vector $\vec{v}$ pointing from the robot's center to the target waypoint. To determine the necessary rotational command, the algorithm calculates the angular error θ_{error} between the robot's current heading $\vec{h}$ and the desired vector $\vec{v}$. This is computed efficiently using the dot product and cross product of the normalized vectors:

$$\vec{h} \cdot \vec{v} = h_x v_x + h_y v_y$$

$$\vec{h} \times \vec{v} = h_x v_y - h_y v_x$$

$$\theta_{error} = \arctan 2\left(\vec{h} \times \vec{v}, \vec{h} \cdot \vec{v}\right)$$

A 45-degree deadzone is implemented to prevent control jitter. If $|\theta_{error}| \leq 45°$, an HTTP GET request for "FORWARD" is transmitted. If the error exceeds these bounds, "LEFT" or "RIGHT" commands are issued. Furthermore, the algorithm actively superimposes collision vectors and edge-margin boundaries, overriding the waypoint vector to steer the robots away from physical boundaries and prevent multi-agent collisions.

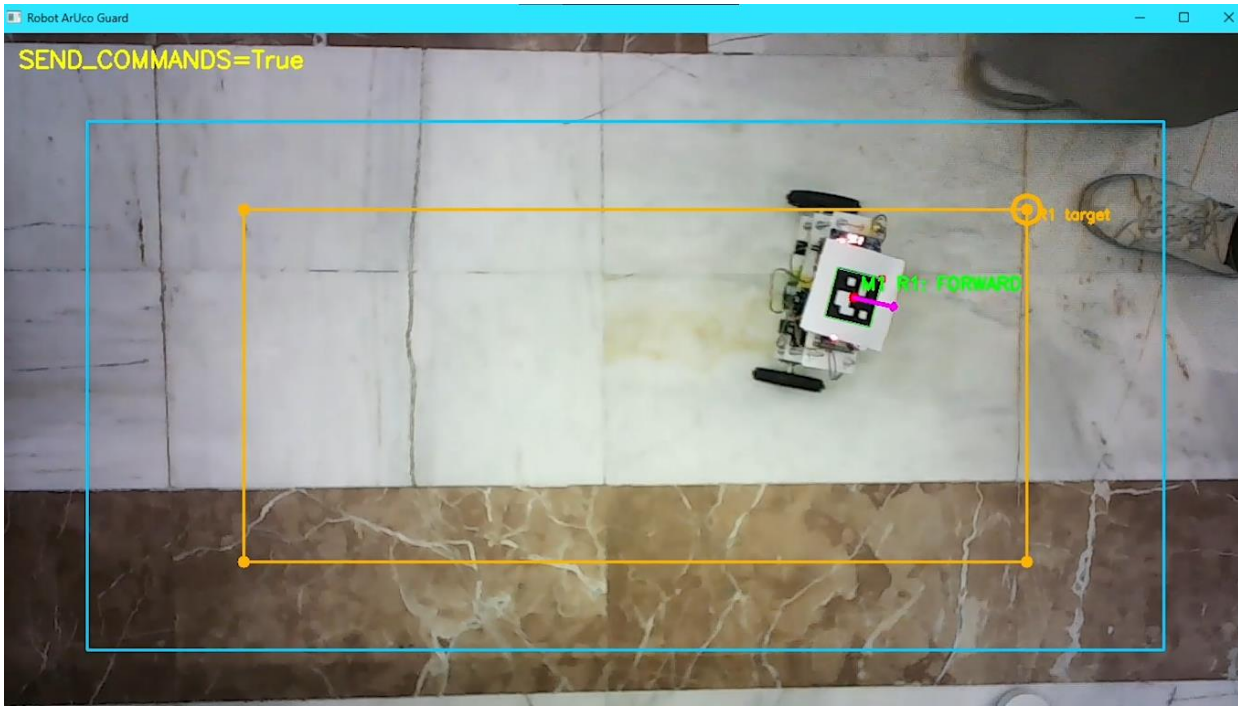

Fig. 7: OpenCV pathfinding interface calculating heading vectors and distance parameters to autonomously route the swarm through designated waypoints.

B. Kinematic Hand-Gesture Teleoperation

In addition to autonomous waypoint navigation, the system integrates a teleoperation module utilizing the MediaPipe machine learning framework. The algorithm extracts 21 3D skeletal landmarks from the operator's hand in real time. Rather than relying on complex gesture classification models, the control logic is highly optimized using a geometric y-coordinate comparison between the tips (landmarks 8, 12, 16, 20) and the Proximal Interphalangeal (PIP) joints (landmarks 6, 10, 14, 18).

Because image pixel coordinates originate from the top-left, a finger is classified as "raised" if the y-coordinate of its tip is less than the y-coordinate of its corresponding PIP joint. The system sums the number of raised fingers to dictate the kinematic state: one finger commands "FORWARD", two fingers command "BACKWARD", three fingers command "RIGHT", and four fingers command "LEFT". This state is sampled, rate-limited to a 0.2-second period to prevent network flooding, and transmitted to the Bridge MCU. Example of these gestures are shown in figure 8 below.

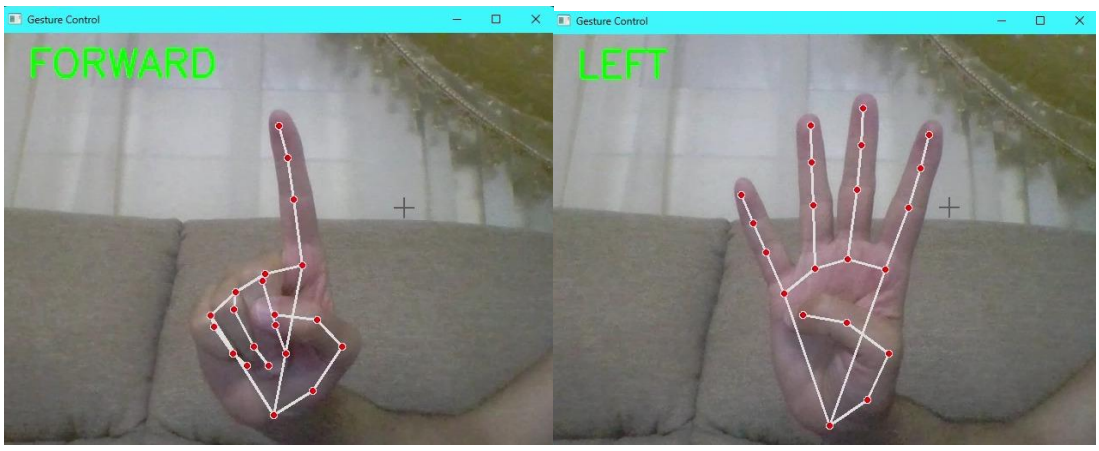

Fig. 8: MediaPipe hand-tracking algorithm successfully mapping 21 skeletal landmarks to trigger real-time directional kinematic commands (Forward and Left).

VII. Ethical, Professional, and Societal Impact

A. Professional Code of Ethics

In the development of autonomous mechatronic systems, adherence to professional ethics is crucial to ensure responsible and trustworthy engineering practices. This project strictly aligns with the core principles of the IEEE Code of Ethics, particularly the mandate "to hold paramount the safety, health, and welfare of the public" and "to disclose promptly factors that might endanger the public or the environment." By applying these principles, the design process for Project S.P.L.I.T. prioritized risk mitigation, transparent data handling, and fail-safe control mechanisms.

B. Ethical and Professional Issues in System Design

Designing a swarm of fast-moving, heavy robotic pendulums introduces several critical safety and ethical considerations. First, the physical safety of human workers sharing the warehouse floor is paramount. A software crash in the PID loop could result in the robot violently falling and acting as a high-speed projectile. Second, the power system utilizes high-discharge Lithium Polymer (LiPo) batteries. Without proper voltage protection, these cells pose a severe fire hazard. Finally, from a data privacy standpoint, the implementation of overhead computer vision cameras necessitates ethical data management to ensure that video feeds are processed locally and securely, rather than transmitting sensitive employee footage to unsecured external servers.

C. Global, Economic, Environmental, and Societal Judgments

To address these ethical considerations, the engineering design was evaluated across four distinct contexts:

- **Global Impact:** The inverted pendulum swarm architecture offers a highly scalable framework for international supply chains. By standardizing this compact technology, global logistics hubs can adopt higher-density storage solutions, significantly reducing the land area required for new industrial facilities.
- **Economic Impact:** The system is designed to be highly cost-effective. By utilizing open-source ESP32 microcontrollers, off-the-shelf L298N drivers, and 3D-printed chassis components rather than proprietary industrial hardware, the economic barrier to entry is drastically lowered. This allows Small and Medium Enterprises (SMEs) to implement automated logistics without multi-million-dollar capital investments.

- **Environmental Impact:** To mitigate environmental harm, the robots are engineered for low power consumption. Trading four wheels for two reduces the friction coefficient, resulting in less electrical energy required for locomotion. However, the use of LiPo batteries presents an E-waste challenge. To address this ethically, the design incorporates a modular battery bay for safe extraction, and future iterations must prioritize the use of readily recyclable battery chemistries and lead-free PCBs.
- **Societal Impact:** Societally, this robotic swarm benefits the workforce by removing humans from ergonomically hazardous material-handling tasks. Furthermore, the intuitive MediaPipe hand-gesture control ensures that the technology is highly accessible, allowing operators of varying technical backgrounds to manage complex robotic fleets efficiently and safely.

VIII. Results & System Validation

The physical implementation of the distributed dual-MCU architecture successfully eliminated network-induced processing pauses. By operating on an isolated Drive MCU, the 200 Hz cascaded PID controller was able to continuously execute its calculations. Fused with the Kalman filter, the system achieved near-zero pitch error, completely neutralizing high-frequency motor vibrations and locking the robots in dynamic equilibrium. Furthermore, the dynamic gripping system proved that a self-balancing inverted pendulum can maintain stability during abrupt center-of-mass shifts, allowing the servo grippers to manipulate payloads flawlessly without tipping over. The telemetry data was successfully transmitted in the web dashboard shown in figure 9 below, proved effective in tuning the PID controllers and saving them to EEPROM on esp32 MCU.

Fig. 9: dashboard and telemetry graphs rendering.

The communication topology proved highly robust under operational loads. The 500,000 baud hardware UART serial connection successfully bridged the Cortex and Drive microcontrollers without floating-point data corruption. Externally, the 2.4 GHz ESP-NOW protocol and WebSockets connections delivered uninterrupted 20 frames-per-second telemetry to the graphical dashboard, validating the theoretical electromagnetic wave models.

IX. Conclusion

Project SPLIT successfully demonstrated that high-density logistics can be achieved without the spatial penalties of traditional four-wheeled rovers. The deployment of a distributed hardware architecture, paired with mathematically optimized sensor fusion and computer vision, yielded a highly functional industrial prototype. Due to the rapid two-week prototyping timeframe, specific engineering compromises were required. Future hardware iterations will replace the archaic L298N motor drivers with highly efficient MOSFET-based alternatives or stepper motors to provide the massive low-end torque needed for incline balancing. Additionally, custom-printed circuit boards will be fabricated to eliminate jumper wire grounding issues that occasionally caused the I2C bus to freeze. The ultimate goal for the swarm remains the implementation of the original mechanical design, a system where two independent two-wheeled robots magnetically dock back-to-back to form a heavy-lifting rover, and subsequently detach using a scissor mechanism to perform distributed micro-deliveries.

References

[1] H. Juang and K. Lum, Design and Control of a Two-Wheel Self-Balancing Robot using the Arduino Microcontroller Board, 10th IEEE International Conference on Control and Automation, 2013.

[2] O. Jamil, M. Jamil, Y. Ayaz, and K. Ahmad, Modeling, Control of a Two-Wheeled Self-Balancing Robot, International Conference on Robotics and Emerging Allied Technologies in Engineering, 2014.

[3] O. M. Mostafa et al., Mechatronics Project: Self-Balance Robot, Helwan University, 2023.

[4] J. Zhao, J. Li, and J. Zhou, Research on Two-Round Self-Balancing Robot SLAM Based on the Gmapping Algorithm, Sensors, vol. 23, 2023.

[5] K. Nakamura, O. Sullivan, and E. Carmody, Development of a 4-DOF Serial Robotic Manipulator for Pick-and-Place Applications, 2025.

[6] W. Harris et al., BiQu: Stair Climbing Robot Dog, Worcester Polytechnic Institute, 2025.

[7] F. T. Ulaby and U. Ravaioli, Fundamentals of Applied Electromagnetics, 8th ed. Pearson, 2020.

[8] Microprocessor & Microcontroller Course: Project Report Guidelines, EE 326 Guidelines.

[9] J. Fang, The LQR Controller Design of Two-Wheeled Self-Balancing Robot Based on the Particle Swarm Optimization Algorithm, Mathematical Problems in Engineering, vol. 2014, 2014.

[10] L. Sun and J. Gan, Researching Of Two-Wheeled Self-Balancing Robot Base On LQR Combined With PID, IEEE, 2010.

[11] H. Zhang and N. Mohamad Nor, Control Strategies for Two-Wheeled Self-Balancing Robotic Systems: A Comprehensive Review, Robotics, vol. 14, 2025.